

Unity Media3 Android Library

Fat AAR Developer Plan

Architecture, module layout, export pipeline, packaging strategy, milestones, testing plan, and implementation guidance for a Unity-facing Android media editing library built on Media3 Transformer.

Prepared for developer implementation · Generated March 10, 2026

Scope

This document describes the Android side only: a Kotlin-based Android library distributed to Unity as a fused or “fat” AAR. Unity UI, editor UX, and C# timeline tooling are intentionally outside scope.

Executive decision

Build a JSON-driven Android export engine behind one small facade class. Keep Media3 internals hidden from Unity. Ship a fused AAR that explicitly packages the required Media3 modules and your own submodules. Support sequential editing, per-item trim and effects, image clips, background music, and dip-to-black transitions in v1. Do not design v1 around true overlapping crossfades or full Premiere-style multi-layer compositing.

1. Executive Summary

The recommended product is an Android library for Unity, written in Kotlin, whose primary job is to transform an abstract timeline description into a Media3 export job. Unity acts as the editor front end. The Android library acts as the media engine.

The best Unity-facing boundary is a single facade class that accepts and returns JSON strings. This matches Unity’s supported bridge types well and avoids exposing Android or Media3 classes over JNI [R5][R6].

Packaging should target a single distributable AAR. Because direct AAR consumption does not resolve transitive dependencies for the consumer, and because the Fused Library plugin does not package transitive dependencies automatically, every required Media3 artifact must be explicitly included in the fused output [R2][R3].

Media3 Transformer is a strong fit for trim, mute, per-item effects, image clips, sequential concatenation, and a parallel music track. However, Media3 Composition still lists crossfading video or audio tracks as unsupported, so v1 should implement a dip-to-black strategy instead of true clip overlap transitions [R1][R4].

2. Confirmed Platform Facts

Unity can call Java and Kotlin plug-in code from C# using Android JNI helper APIs such as `AndroidJavaClass` and `AndroidJavaObject` [R5].

Unity’s supported Java/Kotlin ↔ C# bridge types include primitives, strings, arrays, and JSON-serialized objects. Language-specific collections such as `List`, `Set`, and `Map` are explicitly not supported as a direct boundary [R6].

Android libraries build as AAR artifacts, and Unity supports importing Android Archive plug-ins [R3][R7].

The Android Fused Library plugin is the official way to publish multiple Android libraries as one AAR. Its current prerequisite is AGP 9.0 or higher [R2].

The fused packaging model does not automatically bring along transitive dependencies. Anything that must be present in the final AAR must be declared explicitly in the fused module [R2].

Media3 Transformer documentation describes building Medialtem, EditedMedialtem, sequences, and a Composition, then exporting through Transformer [R4].

Media3 supports clipping, removing audio, image input with duration and frame rate, and ordered audio/video effects. Internally, video effects become GL shader programs [R4].

If multiple Media3 modules are used, they must all use the same version [R8].

AAR libraries should package consumer keep rules through consumerProguardFiles [R9].

3. Product Scope

In scope for this Android library

- Sequential video and image timeline export.
- Per-clip trim, mute, and volume.
- Background music lane running in parallel to the visual sequence.
- Color controls and effect stacks.
- LUT pipeline support, with the expectation that .cube parsing will be your code, not a Unity boundary type.
- Watermark and sticker overlays.
- Dip-to-black and fade-from-black style transitions implemented by timeline expansion rather than true temporal overlap.
- One fused AAR for Unity consumption.
- Structured job state, progress, cancellation, and error reporting.
- A standalone Android demo app for testing outside Unity.

Out of scope for v1

- A full Android preview UI or timeline widget.
- Premiere-style arbitrary overlapping video layers.
- True cross-dissolve transitions between adjacent clips.
- Exposing Media3 classes or Android framework objects directly to Unity.
- iOS, desktop, or visionOS implementations.
- Native C++ unless a later optimization requires it.

4. Core Architecture

The library should be designed as a facade + core engine + packaging system.

Layer	Responsibility	Key rule
Unity C#	Collects user edits, builds project JSON, polls job state, owns editor UI.	Do not embed Media3 assumptions in Unity UI mode.
Android Facade API	Receives JSON, validates requests, starts jobs, returns state as JSON.	Keep public surface tiny and stable.
Core Planner	Normalizes timeline, validates unsupported combinations, expands fake transitions into sequential segments.	No Android UI dependencies.
Effects Layer	Builds RGB matrix, LUT, overlay, and fade effects.	Hide shader/effect details behind simple specs.
Export Layer	Constructs Media3 items, sequences, composition, transformer, and lifecycle handling.	Own Media3 usage here only.

Fused Packaging	Produces one distributable AAR with explicit dependencies included.	No missing transitive artifacts at Unity integration time.
-----------------	---	--

Recommended execution flow:

- Unity serializes the project into JSON.
- Unity calls a single Kotlin facade method, for example `startExport(projectJson)`.
- The facade converts JSON into internal immutable models.
- The planner validates features and expands the user timeline into a Media3-compatible execution plan.
- The export layer builds Media3 objects and starts Transformer.
- Unity polls the job by ID until the export completes, fails, or is canceled.

5. Why the Boundary Should Be JSON

Using JSON at the Unity ↔ Android boundary is the most robust choice for this project.

- Unity officially supports JSON-serialized objects as a bridge type, while language-specific collections are unsupported directly [R6].
- A JSON boundary isolates Unity from internal Android and Media3 model churn.
- Versioning becomes simple: add a document version and keep backward compatibility rules in one place.
- Testing is easier because the same JSON can be replayed in a pure Android demo app.
- Crash reports and bug reproduction improve because the project payload can be logged or persisted as a single object.

Publicly exposing Media3 classes would couple the Unity project to Android implementation details and would create a fragile bridge that is harder to version and harder to debug.

6. Recommended Module Layout

Module	Purpose	Notes
:editor-model	Pure data models and serialization DTOs.	No Android UI; keep portable and stable.
:editor-api	Unity-facing facade and JSON entry points.	The only module Unity should conceptually know.
:editor-core	Validation, timeline normalization, planning, transition expansion.	No direct Media3 code if possible.
:editor-effects	Effect builders for color, LUT, overlays, fades.	Can contain custom effect adapters.
:editor-export	Media3 object construction and export lifecycle management.	Owns Transformer and Composition usage.
:editor-demoapp	Internal Android test app for manual QA and sample jobs.	Essential for debugging outside Unity.
:editor-fused	Distribution-only fused/fat AAR module.	Explicitly includes Media3 artifacts and internal modules.

Key principle: Unity should never call deep internals. It should only call the facade and interpret JSON results.

7. Suggested Public API

Expose a minimal job-oriented API. Use strings and primitives only.

```
object UnityVideoEditor {
    @JvmStatic fun initialize(configJson: String): String
    @JvmStatic fun validateProject(projectJson: String): String
    @JvmStatic fun startExport(projectJson: String): String
}
```

```

@JvmStatic fun getJobState(jobId: String): String
@JvmStatic fun cancelJob(jobId: String): String
@JvmStatic fun releaseJob(jobId: String): String
@JvmStatic fun releaseAll(): String
}

```

Recommended result shapes:

- initialize returns version info, supported features, and configuration status.
- validateProject returns errors, warnings, normalized duration, and any downgraded features.
- startExport returns a job ID immediately.
- getJobState returns lifecycle state, progress percent, output URI, and structured error fields if needed.
- cancelJob returns whether cancellation was requested or the job already finished.

Prefer polling in v1. Callback interfaces can be added later if needed, but they increase bridge complexity and failure cases.

8. Project JSON Model

The project document should be versioned and explicit. A practical model is shown below.

```

{
  "version": 1,
  "assets": [
    { "id": "v1", "type": "video", "uri": "content://..." },
    { "id": "img1", "type": "image", "uri": "content://..." },
    { "id": "a1", "type": "audio", "uri": "content://..." }
  ],
  "videoTrack": [
    {
      "assetId": "v1",
      "trimStartMs": 1000,
      "trimEndMs": 8000,
      "removeAudio": false,
      "volume": 0.9,
      "effects": {
        "brightness": 0.05,
        "contrast": 1.1,
        "saturation": 1.0,
        "lut": null,
        "overlayIds": ["logo1"]
      },
      "transitionOut": {
        "type": "dip_to_black",
        "durationMs": 400
      }
    },
    {
      "assetId": "img1",
      "durationMs": 3000,
      "frameRate": 30,
      "transitionIn": {
        "type": "fade_from_black",
        "durationMs": 400
      }
    }
  ],
  "audioTracks": [

```

```

    {
      "assetId": "a1",
      "trimStartMs": 0,
      "trimEndMs": 12000,
      "loop": false,
      "volume": 0.5
    }
  ],
  "overlays": [
    {
      "id": "logo1",
      "uri": "content://...",
      "x": 0.85,
      "y": 0.10,
      "scale": 0.20
    }
  ],
  "export": {
    "outputUri": "content://...",
    "videoMimeType": "video/avc",
    "audioMimeType": "audio/mp4a-latm",
    "width": 1920,
    "height": 1080,
    "fps": 30
  }
}

```

Version the JSON now, even if only version 1 exists at launch.

9. Internal Export Pipeline

The planner and export engine should follow a deterministic pipeline.

- Step 1: Parse. Convert JSON into immutable internal models and reject malformed or missing fields.
- Step 2: Validate assets. Confirm readable input URIs and a writable output target.
- Step 3: Normalize timeline. Sort clips, fill defaults, clamp trim ranges, and compute expected duration.
- Step 4: Expand transitions. Convert dip-to-black or fade-from-black requests into sequential clip segments rather than temporal overlap.
- Step 5: Build Media3 media items. Apply clipping, image duration, remove-audio, frame rate, and effect stacks [R4].
- Step 6: Build sequences. Create one main visual sequence and one or more audio sequences, then combine them in a Composition [R1].
- Step 7: Configure Transformer. Set output format, listener, cancellation, and progress behavior [R8].
- Step 8: Execute export. Track state in an internal job registry.
- Step 9: Return state to Unity. Expose progress, warnings, output location, and structured errors.

10. Transition Strategy

Do not base v1 on true crossfades. Media3's official multi-asset composition documentation still lists crossfading video or audio tracks as unsupported [R1].

Recommended v1 strategy:

- Implement fade-out to black on the tail of clip A.
- Implement fade-in from black on the head of clip B.

- Treat them as two independent sequential effects, not overlapping playback.
- Represent these operations in the planner by splitting clips into logical segments if needed.

This yields a professional-looking result with lower complexity and fits the supported sequential sequence model well.

11. Effects and Visual Processing

Media3 supports ordered audio and video effects and turns video effect chains into GL shader programs [R4]. This is enough for a solid v1 effects layer.

Feature	Implementation direction
Brightness / contrast / saturation	Expose as simple scalar fields in JSON; map to effect builders or RGB matrix transforms.
3D LUT	Parse .cube in your own code, validate dimensions, then adapt into a Media3-friendly LUT/effect pipeline.
Watermark / sticker overlay	Represent as overlay specs with position, scale, opacity, and z-order if needed.
Fade to black / from black	Use a brightness or matrix ramp effect over a short segment.
Future preview reuse	Keep effect builders reusable so the same specs can later drive preview, not just export.

Important: treat .cube ingestion as your own feature layer. Do not depend on Unity passing raw complex effect types over the bridge.

12. Audio Architecture

The audio design should stay simple in v1.

- Each visual clip may either keep or remove its original audio [R4].
- Each visual clip should have its own volume scalar.
- A background music track should be modeled as a separate audio sequence in the Composition [R1].
- Trim and optional looping should be handled before sequence construction.
- If multiple audio-only tracks are added later, keep them as explicit future scope so the JSON model can expand cleanly.

13. Packaging as a Fat / Fused AAR

Your distribution target is one library artifact that Unity can import directly. The official Android guidance for publishing multiple Android libraries as one artifact is the Fused Library plugin, which currently requires AGP 9.0 or higher [R2].

This is the correct release strategy for your use case, but two constraints matter:

- Direct AAR consumption is inconvenient when dependencies are spread across multiple artifacts [R3].
- The fused packaging step does not automatically pull in transitive dependencies, so every Media3 artifact you need must be declared in the fused module explicitly [R2].

Recommended fused module contents:

- Your internal modules: :editor-model, :editor-api, :editor-core, :editor-effects, :editor-export.
- Explicit Media3 modules you use, such as media3-transformer, media3-effect, and media3-common [R8].
- Any additional AndroidX or support artifacts that the fused output must contain, if your chosen packaging setup requires explicit inclusion.

Distribution rule: the Unity consumer should not need to manually add extra Media3 AARs.

14. Build System and Versioning

- Use one Android multi-module Gradle project during development.
- Pin all Media3 modules to the same version; the Media3 docs explicitly require version alignment across modules [R8].
- Prefer release and debug library variants for internal testing.
- Package consumer rules with consumerProguardFiles in AAR outputs [R9].
- Keep a machine-readable plugin version in the Android facade, for example semantic versioning with a minimum supported project JSON version.

15. Threading, Lifecycle, and Reliability

Treat export as a managed background operation inside the library. The public API should be synchronous only for lightweight validation calls and asynchronous in behavior for export jobs.

Concern	Recommended rule
Job registry	Maintain an internal map keyed by jobId with immutable snapshots of state.
Cancellation	Expose idempotent cancellation. Return success if already canceled or already finished.
Progress	Expose a normalized 0–100 or 0.0–1.0 value. Prefer polling in v1.
Errors	Return structured error codes and a developer-readable message, plus root cause class if safe.
Resource cleanup	Release Media3 and file handles promptly when jobs finish or fail.
Thread confinement	Keep facade methods thin; dispatch heavy planning and export work off the Unity call site thread.

17. Main Risks and Mitigations

Risk	Why it matters	Mitigation
Crossfade expectations	Product owners may expect true NLE overlap transitions.	Document clearly that v1 uses dip-to-black because Media3 Composition does not support crossfade transitions [R1].
Bridge complexity	Large JNI surfaces become fragile and slow to evolve.	Use one facade and JSON only [R5][R6].
Dependency packaging	A normal AAR can leave consumers missing artifacts.	Ship a fused AAR and explicitly include required dependencies [R2][R3].
Effect portability	Custom overlays and LUT paths can expose device-specific issues.	Build feature flags and validation warnings; test on representative devices.
Preview temptation	Trying to solve preview and export together can stall the project.	Ship export engine first, preview later.
Version drift	Mismatched Media3 artifacts can cause integration failures.	Pin all Media3 modules to one version [R8].

18. Definition of Done

- A single fused AAR can be imported into Unity without adding extra Media3 artifacts manually.
- Unity can start an export by passing JSON and can monitor completion by polling job state.

- Sequential video and image timelines export correctly.
- Background music works in parallel with the main visual sequence.
- Per-item trim, mute, and volume are respected.
- Watermark overlays and basic color controls work on supported devices.
- Dip-to-black transitions render correctly.
- Errors are structured, stable, and useful for developers.
- The Android demo app passes the same core scenarios as Unity integration.
- Release builds package consumer rules appropriately for downstream shrinking and obfuscation [R9].

19. Implementation Recommendations

- Use Kotlin for the library. Do not add native C++ unless profiling later proves it is necessary.
- Start with export only. Keep preview and editor UI out of the Android library until export reliability is proven.
- Persist the last export request for easier debugging and reproduction.
- Make every public response include the library version and a schema version when practical.
- Keep logs developer-friendly but sanitize any sensitive path details before exposing them to Unity logs.
- Reserve room in the JSON schema for future audio track expansion, preview modes, and hardware capability probing.

20. Final Recommendation

Proceed with a Kotlin-based Android export engine distributed as a fused AAR for Unity. Use one JSON facade, a planner that expands unsupported transition ideas into supported sequential segments, and a module structure that isolates public API, planning logic, effects, export, and packaging. This is the most stable path to a developer-usable library with room to grow.

References

[R1] Android Developers. “Multi-asset editing.” Notes that sequences in a Composition are subject to limitations and that crossfading video or audio tracks is not yet supported.

developer.android.com/media/media3/transformer/multi-asset

[R2] Android Developers. “Publish multiple Android libraries as one with Fused Library.” States AGP 9.0+ is required and describes fused library packaging. developer.android.com/build/publish-library/fused-library

[R3] Android Developers. “Create an Android library.” Explains Android library modules and AAR outputs. developer.android.com/studio/projects/android-library

[R4] Android Developers. “Transformations.” Documents clipping, remove-audio, image input duration/frame rate, effects, and GL shader based effect processing.

developer.android.com/media/media3/transformer/transformations

[R5] Unity Manual. “Calling Java and Kotlin plug-in code from C# scripts.” Documents `AndroidJavaClass` / `AndroidJavaObject` bridge usage. docs.unity3d.com/6000.1/Documentation/Manual/android-plugins-java-code-from-c-sharp.html

[R6] Unity Manual. “Supported data types for Java/Kotlin and C# code.” States supported primitive/string/array/JSON types and unsupported language-specific collections. docs.unity3d.com/6000.3/Documentation/Manual/android-datatypes-java-kotlin-csharp.html

[R7] Unity Manual. “Import an Android Archive plug-in.” Covers AAR import into Unity. docs.unity3d.com/2023.2/Documentation/Manual/android-aar-import.html

[R8] Android Developers. “Getting started.” Shows Media3 Transformer setup, example artifacts, and the requirement that all Media3 modules use the same version.

developer.android.com/media/media3/transformer/getting-started

[R9] Android Developers. “Optimization for library authors.” Recommends packaging consumer rules using consumerProguardFiles for AAR libraries. developer.android.com/topic/performance/app-optimization/library-optimization